

Lucas Augusto da Silva Cardoso (202200332300)

DCA3703 | Programação Paralela | Relato de experimento

TAREFA 11: SIMULAÇÃO DA EQUAÇÃO DE NAVIER-STOKES SIMPLIFICADA COM DISCRETIZAÇÃO POR DIFERENÇAS FINITAS

Este documento detalha a abordagem metodológica e a solução implementada para o Experimento Computacional proposto na disciplina de Programação Paralela, em 08/09/2026.

1. EXPERIMENTO

Escreva um código que simule o movimento de um fluido ao longo do tempo usando a equação de Navier-Stokes, considerando apenas os efeitos da viscosidade. Desconsidere a pressão e quaisquer forças externas. Utilize diferenças finitas para discretizar o espaço e simule a evolução da velocidade do fluido no tempo. Inicialize o fluido parado ou com velocidade constante e verifique se o campo permanece estável. Em seguida, crie uma pequena perturbação e observe se ela se difunde suavemente. Após validar o código, paralelize-o com OpenMP e explore o impacto das cláusulas schedule e collapse no desempenho da execução paralela.

2. CONCEITOS RELEVANTES

- **Cláusula schedule:**
  - **static (divisão justa)**: é o gerente de uma empresa que divide a lista de tarefas igualmente antes de começar. É ótimo se todas as tarefas levarem o mesmo tempo.

- ➢ **dynamic (fila de senha):** cada trabalhador pega 1 tarefa. Assim que termina, volta ao gerente e pede a próxima. Perfeito se algumas tarefas forem muito mais pesadas que outras.
- ➢ **guided (O lote regressivo)**: Começa entregando pacotes grandes de tarefas e vai diminuindo o tamanho dos pacotes conforme o trabalho acaba. Balanceia a carga sem fazer o gerente falar toda hora.
- ➢ **chunksize (Tamanho do pacote)**: Quantas tarefas o gerente entrega por vez (ex: "pegue de 10 em 10").

❖ **Cláusula collapse:** É a junção de listas. Se você tem loops dentro de loops (uma matriz, por exemplo), o loop de fora pode ter poucas tarefas para todo mundo. O collapse junta tudo numa lista gigante só, para ninguém ficar de braços cruzados.

3. MÁQUINA UTILIZADA

Para a execução deste experimento, estou utilizando uma máquina equipada com um processador AMD Ryzen 7 5700X (8 núcleos físicos e 16 threads), que é o componente central para a avaliação das diretivas OpenMP.

4. NAVIER-STOKES

A Equação de Navier-Stokes é o conjunto de equações matemáticas que descreve como os fluidos (líquidos e gases) se movimentam. Na prática, ela é simplesmente a Segunda Lei de Newton (F = ma) aplicada a um fluido continuous.

A equação calcula como a velocidade do fluido muda ao longo do tempo, equilibrando as forças que atuam sobre ele:

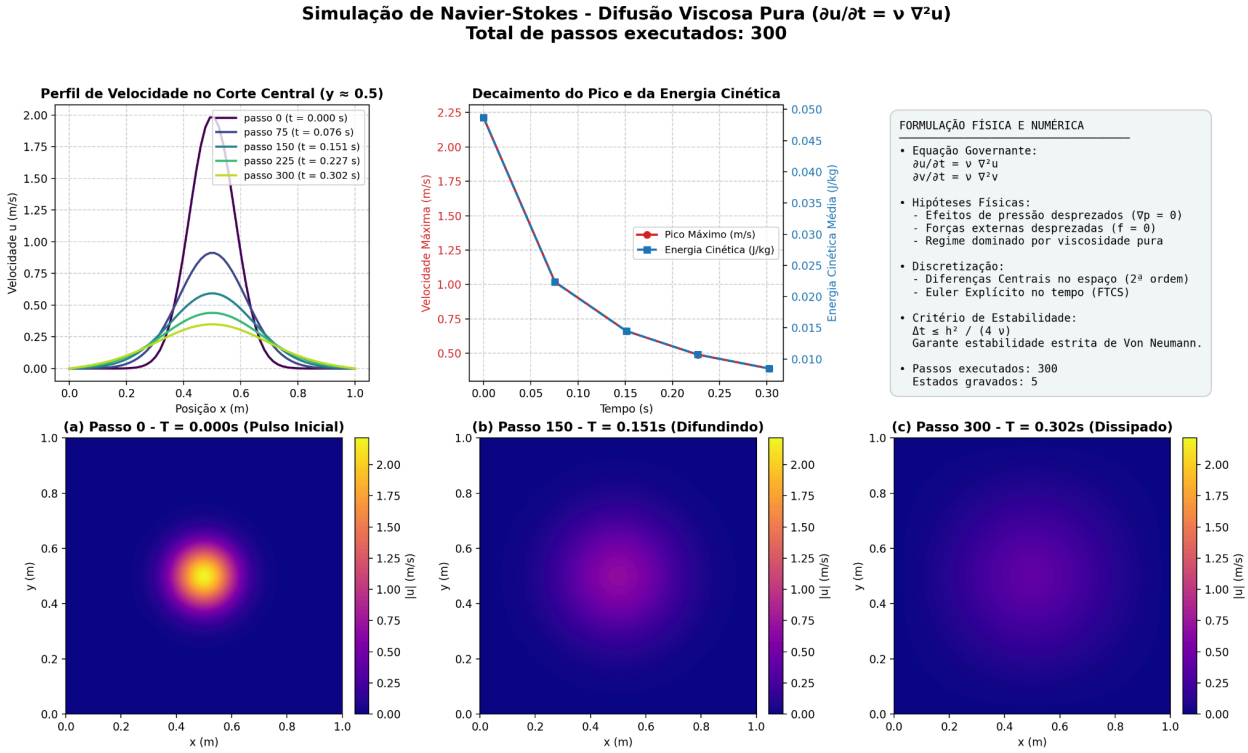
$$\underbrace{\rho\left(\frac{\partial \mathbf{u}}{\partial t} + \mathbf{u}\cdot\nabla\mathbf{u}\right)}_{\text{Massa}\times\text{Aceleração}} = \underbrace{-\nabla p}_{\text{Pressão}} + \underbrace{\mu\nabla^2\mathbf{u}}_{\text{Viscosidade}} + \underbrace{\mathbf{f}}_{\text{Forças Externas}}$$

- Aceleração (Esquerda): Como a velocidade do fluido muda no tempo e no espaço (movimento do próprio fluido).
- Pressão: O fluido é "empurrado" de zonas de alta pressão para zonas de baixa pressão.

- Viscosidade: O "atrito" interno do fluido. É o que faz o mel escorrer devagar e a perturbação da água sumir suavemente com o tempo.
- Forças Externas: Ação da gravidade, campos magnéticos, etc.

É uma equação diferencial não linear altamente complexa. Na maioria das vezes, não existe uma solução matemática exata, sendo necessário usar computadores para aproximar os resultados.

Figura 1: Plotagem 2D do código de Navier-Stokes



5. CÓDIGO

O programa simula o espalhamento de um fluido em uma grade 2D considerando apenas a sua viscosidade (atrito interno). O objetivo principal é calcular a evolução da velocidade do fluido no tempo e testar qual estratégia do OpenMP deixa essa simulação mais rápida na CPU.

Para tal, o programa começa lendo as configurações passadas pelo usuário (ou usa padrões como uma matriz de 256 por 256 e 200 passos no tempo. Ele cria

uma "gota" de velocidade no centro de um fluido parado para observar como ela se espalha e calcula os coeficientes matemáticos que serão usados nos passos seguintes.

Figura 2: static + toda a lógica de código repetida nas outras cláusulas

(O laço externo é dividido estaticamente antes da execução. As linhas da matriz são entregues às threads em blocos fixos de tamanho tamanho_chunk. Possui o menor custo de gerenciamento (overhead))

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

#include "navier_stokes/fluido.h"

#ifndef CHUNK
#define CHUNK 32
#endif

int main(int argc, char *argv[]) {
   int tamanho_malha = (argc > 1) ? atoi(argv[1]) : 256;
   int quantidade_passos = (argc > 2) ? atoi(argv[2]) : 200;
   int quantidade_threads = (argc > 3) ? atoi(argv[3]) : omp_get_max_threads();
   int tamanho_chunk = (argc > 4) ? atoi(argv[4]) : CHUNK;

   omp_set_num_threads(quantidade_threads);

   GradeFluido *grade = criar_grade(
          tamanho_malha,
           tamanho_malha,
           1.0,
          1.0,
          0.05,
          quantidade_threads
   );
   grade->num_passos = quantidade_passos;
   inicializar_perturbacao_gaussiana(grade, 0.0, 0.0, 2.0, 0.08);

   double coef_x = grade->nu * grade->dt / (grade->dx * grade->dx);
```

```
    double coef_y = grade->nu * grade->dt / (grade->dy * grade->dy);
    double * restrict campo_atual = grade->u_atual;
    double * restrict campo_proximo = grade->u_prox;

    printf("TAREFA 11: schedule(static)\n");
    printf("Malha: %dx%d | Passos: %d | Threads: %d | Chunk: %d\n",
           tamanho_malha, tamanho_malha, quantidade_passos,
           quantidade_threads, tamanho_chunk);

    double tempo_inicial = omp_get_wtime();

    for (int passo = 0; passo < grade->num_passos; passo++) {
        #pragma omp parallel for schedule(static, tamanho_chunk)
        for (int i = 1; i < tamanho_malha - 1; i++) {
            for (int j = 1; j < tamanho_malha - 1; j++) {
                int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
                double laplaciano_x = (campo_atual[FLUIDO_IDX(i + 1, j, tamanho_malha)] -
2.0 * campo_atual[indice_centro] + campo_atual[FLUIDO_IDX(i - 1, j,
tamanho_malha)]) * coef_x;
                double laplaciano_y = (campo_atual[FLUIDO_IDX(i, j + 1, tamanho_malha)] -
2.0 * campo_atual[indice_centro] + campo_atual[FLUIDO_IDX(i, j - 1,
tamanho_malha)]) * coef_y;
                double laplaciano = laplaciano_x + laplaciano_y;
                campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
            }
        }

            // esse loop serve para aplicar as condições de contorno, garantindo que a
velocidade nas bordas da malha seja zero.
        aplicar_condicoes_contorno(tamanho_malha, campo_proximo);

        double *ponteiro_temporario = campo_atual;
        campo_atual = campo_proximo;
        campo_proximo = ponteiro_temporario;
    }

    double tempo_total = omp_get_wtime() - tempo_inicial;
    printf("Tempo de execucao: %.6f s | Pico central: %.6f m/s\n",
           tempo_total,
```

```
        campo_atual[FLUIDO_IDX(tamanho_malha / 2, tamanho_malha / 2,
tamanho_malha)]);

    liberar_grade(grade);
    return 0;
}
```

Figura 3: laço paralelizado com dynamic

(os blocos de linhas da matriz são distribuídos dinamicamente sob demanda. Quando uma thread termina seu bloco, ela solicita outro ao runtime).

```
#pragma omp parallel for schedule(dynamic, tamanho_chunk)
    for (int i = 1; i < tamanho_malha - 1; i++) {
        for (int j = 1; j < tamanho_malha - 1; j++) {
            int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
            double laplaciano_x = Mesmo trecho da figura 1
            double laplaciano_y = Mesmo trecho da figura 1
            double laplaciano = laplaciano_x + laplaciano_y;
            campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
    }
}
```

Figura 4: laço paralelizado com guided

(similar ao dinâmico, mas os blocos iniciais entregues às threads são grandes e diminuem gradualmente de tamanho, buscando balancear a carga e reduzir o número de requisições)

```
#pragma omp parallel for schedule(guided, tamanho_chunk)
    for (int i = 1; i < tamanho_malha - 1; i++) {
        for (int j = 1; j < tamanho_malha - 1; j++) {
            int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
            double laplaciano_x = Mesmo trecho da figura 1
            double laplaciano_y = Mesmo trecho da figura 1
            double laplaciano = laplaciano_x + laplaciano_y;
            campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
    }
}
```

Figura 5: laço paralelizado com static + collapse

(a cláusula collapse(2) junta os laços de linhas e colunas em um único laço plano unidimensional. A distribuição é feita de forma estática sobre esse espaço de iterações ampliado, aumentando o volume de tarefas disponíveis para as threads)

```
#pragma omp parallel for collapse(2) schedule(static, tamanho_chunk)
   for (int i = 1; i < tamanho_malha - 1; i++) {
      for (int j = 1; j < tamanho_malha - 1; j++) {
         int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
         double laplaciano_x = Mesmo trecho da figura 1
         double laplaciano_y = Mesmo trecho da figura 1
         double laplaciano = laplaciano_x + laplaciano_y;
         campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
   }
}
```

Figura 6: laço paralelizado com dynamic + collapse

(aplica a distribuição dinâmica sobre o laço unificado gerado pelo collapse, garantindo alocação sob demanda para a matriz inteira)

```
#pragma omp parallel for collapse(2) schedule(dynamic, tamanho_chunk)
  for (int i = 1; i < tamanho_malha - 1; i++) {
      for (int j = 1; j < tamanho_malha - 1; j++) {
         int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
         double laplaciano_x = Mesmo trecho da figura 1
         double laplaciano_y = Mesmo trecho da figura 1
         double laplaciano = laplaciano_x + laplaciano_y;
         campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
   }
}
```

Figura 7: laço paralelizado com guided + collapse

(aplica o escalonamento guiado sobre o laço unificado do collapse, combinando blocos decrescentes com um maior nível de paralelismo)

```
#pragma omp parallel for collapse(2) schedule(guided, tamanho_chunk)
   for (int i = 1; i < tamanho_malha - 1; i++) {
      for (int j = 1; j < tamanho_malha - 1; j++) {
```

```
        int indice_centro = FLUIDO_IDX(i, j, tamanho_malha);
        double laplaciano_x = Mesmo trecho da figura 1
        double laplaciano_y = Mesmo trecho da figura 1
        double laplaciano = laplaciano_x + laplaciano_y;
        campo_proximo[indice_centro] = campo_atual[indice_centro] + laplaciano;
    }
}
```

6. RESULTADOS

Abaixo, é possível visualizar o resultado de desempenho de compilação dos códigos.

Tabela 1: Comparação de desempenho das estratégias adotadas variando chunk

Chunk	V1 static	static + collapse	dynamic	dynamic + collapse	guided	guided + collapse
1	0,012462	0,066059	0,013348	0,794189	0,007822	0,020483
4	0,011436	0,038426	0,010135	0,197290	0,007901	0,018564
8	0,012157	0,036490	0,008200	0,103150	0,008480	0,019344
12	0,010264	0,035682	0,007697	0,075197	0,008348	0,019464
16	0,010925	0,032078	0,007654	0,061570	0,010136	0,019169
20	0,012041	0,031205	0,008922	0,053285	0,007824	0,018603
24	0,009732	0,030081	0,009692	0,048331	0,008925	0,02712
28	0,010002	0,030507	0,01075	0,042177	0,009655	0,025028
32	0,009425	0,031742	0,010757	0,03801	0,009305	0,020075
36	0,009865	0,033581	0,008969	0,03503	0,009123	0,018612
40	0,009731	0,032767	0,010173	0,034792	0,009345	0,018704
44	0,00995	0,036654	0,009376	0,035233	0,00864	0,022187
48	0,01134	0,034672	0,009487	0,032876	0,010109	0,021529
52	0,010361	0,036402	0,010493	0,030151	0,010003	0,022245
56	0,012271	0,031847	0,01194	0,028886	0,009702	0,02175
60	0,011693	0,029405	0,011855	0,032425	0,009395	0,025528
64	0,008311	0,02565	0,012373	0,03203	0,011414	0,022387

Figura 8: Comparação de desempenho variando threads

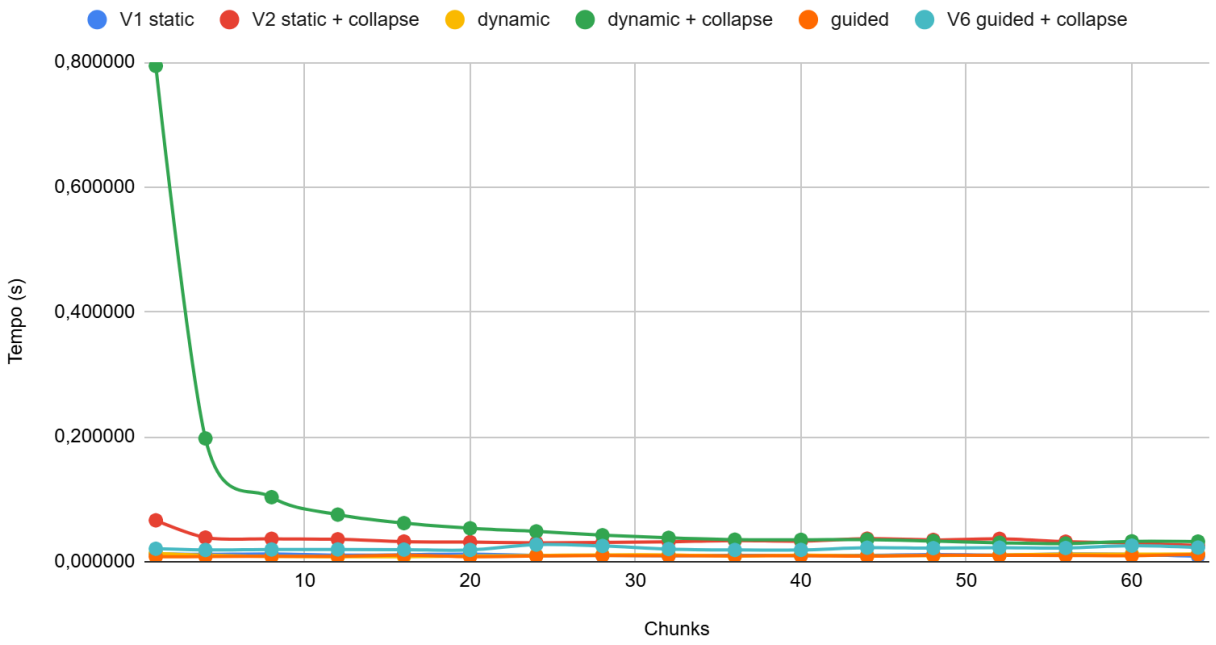


Figura 9: Comparação de desempenho variando chunks (exceto dynamic + collapse)

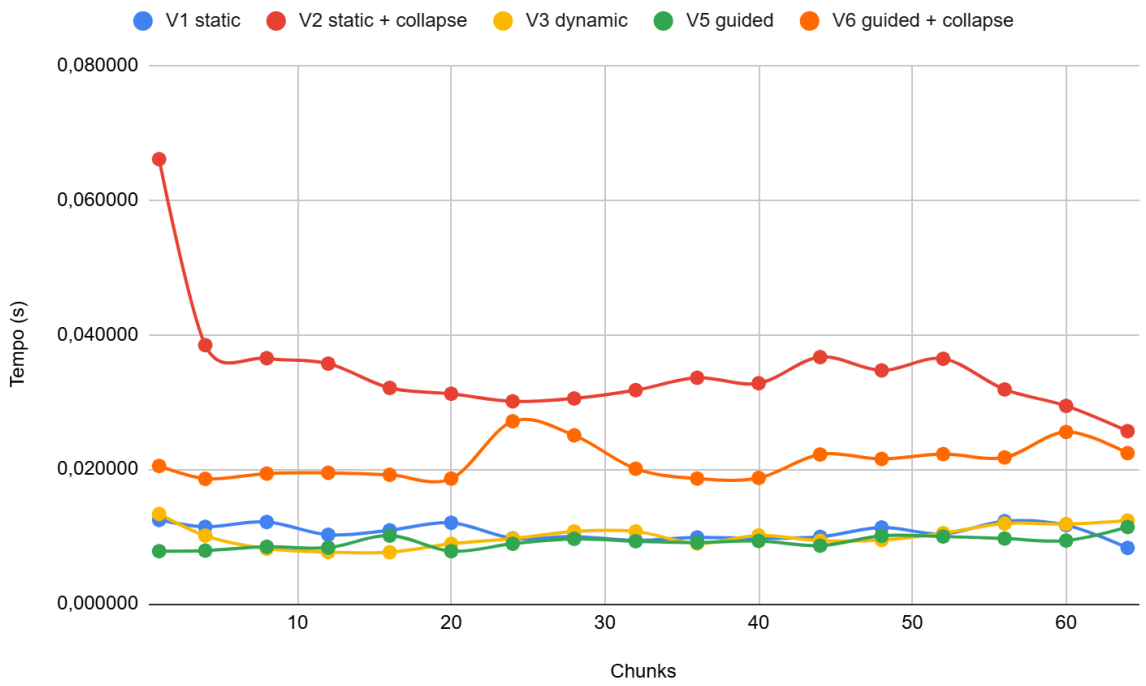
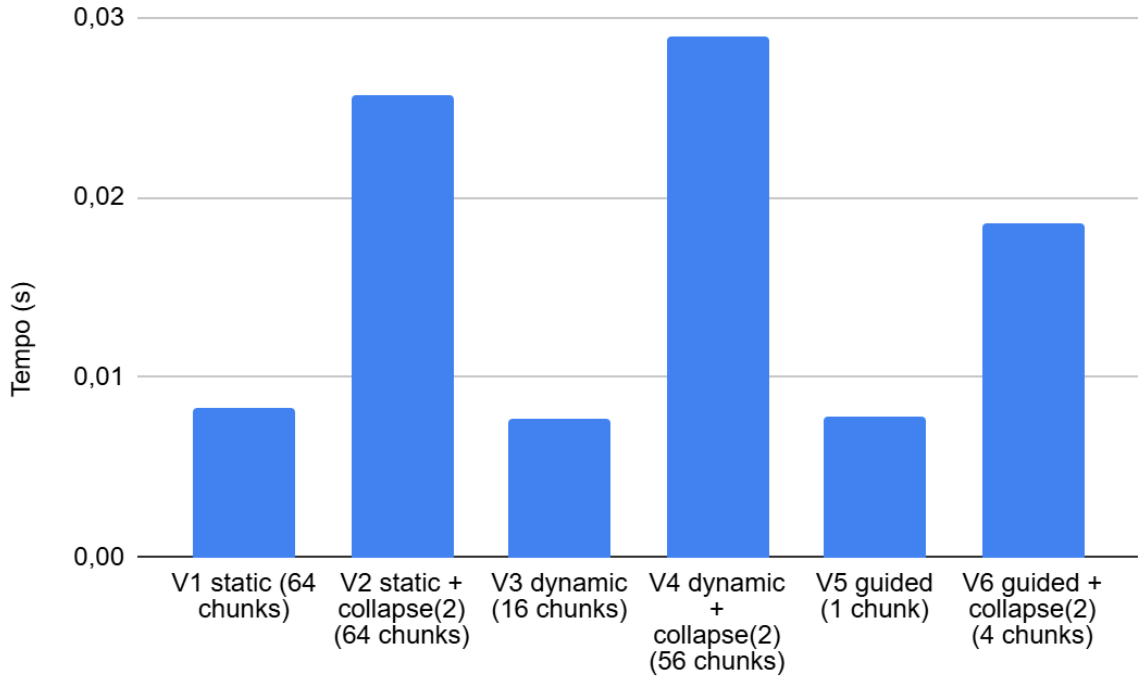


Figura 10: Melhores tempos de cada versão



Ao analisar os dados do experimento podemos perceber que olhando apenas o laço mais externo paralelizado, os três métodos de escalonamento apresentaram os melhores tempos gerais da simulação, mas com comportamentos bem distintos:

- **V5 (guided):** Foi a estratégia mais rápida do teste, atingindo marcas de até 0,0078s.
    - **Por que isso acontece?** O escalonamento guided começa entregando blocos grandes de trabalho para as threads e diminui o tamanho desses blocos gradualmente. Isso garante o melhor dos dois mundos, onde os blocos grandes no início evitam que as threads percam tempo pedindo trabalho à CPU, e os blocos menores no final garantem que todas as threads terminam quase ao mesmo tempo, sem ninguém ficar ocioso.
- **V3 (dynamic):** Obteve o seu melhor tempo em 0,0076 s com chunksize = 16, mas piorou conforme o chunk aumentou.

    - **Por que isso acontece?** No modo dynamic, as threads pedem um novo bloco de trabalho assim que terminam o anterior. Com blocos médios (12 a 20), o trabalho fica bem dividido. Porém, quando o chunksize fica muito grande (40), o comportamento se aproxima do estático, perdendo flexibilidade e aumentando ligeiramente o tempo.
- **V1 (static):** Manteve um tempo praticamente constante ao longo de quase todos os testes (0,009).
    - **Por que isso acontece?** O modo static divide as linhas da matriz de forma fixa antes mesmo de o loop começar. Como cada ponto da nossa malha exige exatamente a mesma quantidade de contas (cálculo de 4 vizinhos), a carga de trabalho de cada linha é perfeitamente igual. Por não haver gerenciamento em tempo de execução, ele apresenta baixíssimo overhead (custo de controle) e entrega um resultado sólido e estável.2.

A inclusão do collapse(2) junta os laços de linha “i” e coluna “j” em uma única lista gigante de iterações (N). Em teoria, isso aumenta o paralelismo. Porém, nos nossos testes, o collapse piorou o desempenho em todas as configurações.

- **V4 (dynamic + collapse)**: Foi a pior combinação do experimento, alcançando 0,7941s com chunksize = 1, cerca de 100 vezes mais lento que os melhores casos.
    - **Por que isso acontece?** Ao unificar os dois loops de uma matriz 256 por 256, criamos 65.536 iterações individuais. Com chunksize = 1 e escalonamento dynamic, o OpenMP precisou gerenciar a entrega de 65.536 pacotes minúsculos de trabalho em tempo de execução. O tempo que a CPU gastou organizando a fila de tarefas e sincronizando as threads (overhead de concorrência) foi imensamente maior do que o tempo gasto calculando a física do fluido. Conforme o chunksize aumentou para 32 ou 64, esse custo caiu drasticamente (chegando a 0,032), pois o número de chamadas ao OpenMP diminuiu.
- **V2 (static + collapse) e V6 (guided + collapse)**: Também ficaram mais lentas que suas versões sem collapse (na faixa de 0,018s a 0,038s).
    - Por que isso acontece? O collapse é excelente quando o laço externo tem poucas iterações (por exemplo, apenas 2 ou 4 linhas para 16 threads). No nosso caso, o laço externo já possuía 256 linhas, o que era

mais do que suficiente para ocupar todas as threads da CPU. Fundir os loops criou um espaço de iterações desnecessariamente grande, gerando custo adicional de índice e divisão de memória sem trazer nenhum benefício real de paralelismo.

7. CONSIDERAÇÕES FINAIS

- **A física do problema é uniforme:** Toda a matriz dá o mesmo trabalho para ser calculada.
- **Menos gerenciamento = Maior velocidade:** Métodos que exigem pouca conversa entre as threads (guided e static sem collapse) vencem porque a CPU gasta 100% do tempo fazendo cálculos, em vez de gerenciar tarefas.
- **collapse não é solução para tudo:** Ele só deve ser usado se o loop externo for pequeno. Em matrizes grandes e equilibradas, ele adiciona uma sobrecarga de processamento que atrasa a execução.